ComicCraft

AI Comic Story Creator using Gemini Models

PROJECT TEAM

Name	Team Role
Dhanushika S (Team Leader)	Documentation
Nandhini V (Team Member)	Coding
Kaviya T (Team Member)	Voice Over
Indumathi R (Team Member)	API Testing Video
Akshaya B (Team Member)	Demo Video

Project Report

ComicCraft - AI Comic Story Creator using Gemini Models

Project Description:

ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations based on user-provided prompts. Built with FastAPI and integrated with Google's Gemini AI models along with Stable Diffusion, ComicCraft streamlines the creative process of generating storylines, dialogues, and vivid comic-style imagery automatically.

The application takes in user information—such as story prompt, main character name, setting, tone, and art style—and generates a panel-by-panel storyline and corresponding illustrations. Users can view the comic preview directly on the web interface and also download the full comic in PDF format, structured with both images and narration.

With a clean, interactive HTML interface and efficient image generation powered by Hugging Face's Diffusers library, ComicCraft ensures that each story feels cohesive and visually engaging. It offers both API endpoints and an intuitive frontend for ease of use, making comic creation accessible even to non-artists and casual users.

Scenarios:

Scenario 1 : A user enters a prompt describing their comic story, such as "A brave fox exploring an enchanted forest." The system uses this input to generate a complete comic strip, including multiple panels with accompanying images and text. The user also specifies preferences like the main character's name, the setting (e.g., forest), tone (e.g., dramatic), and art style (e.g., anime). The system combines **Gemini 1.5 Pro** for story generation and **Stable Diffusion** for image creation to produce a visually immersive, personalized comic strip. This personalized approach enhances user satisfaction by generating a cohesive comic story that fits the user's creative vision.

Scenario 2: A user wants a light-hearted, cartoonish feel for their comic. On the input form, they select "funny" as the tone and "comic book" as the art style (instead of the default or their previous choices). When they submit, the system regenerates the entire pipeline—outline, story, and images—this time with prompts and model instructions tailored to produce a more humorous narrative and a classic comic-book look. This lets users iterate on and customize the mood and visual style of their comic until it matches their creative vision.

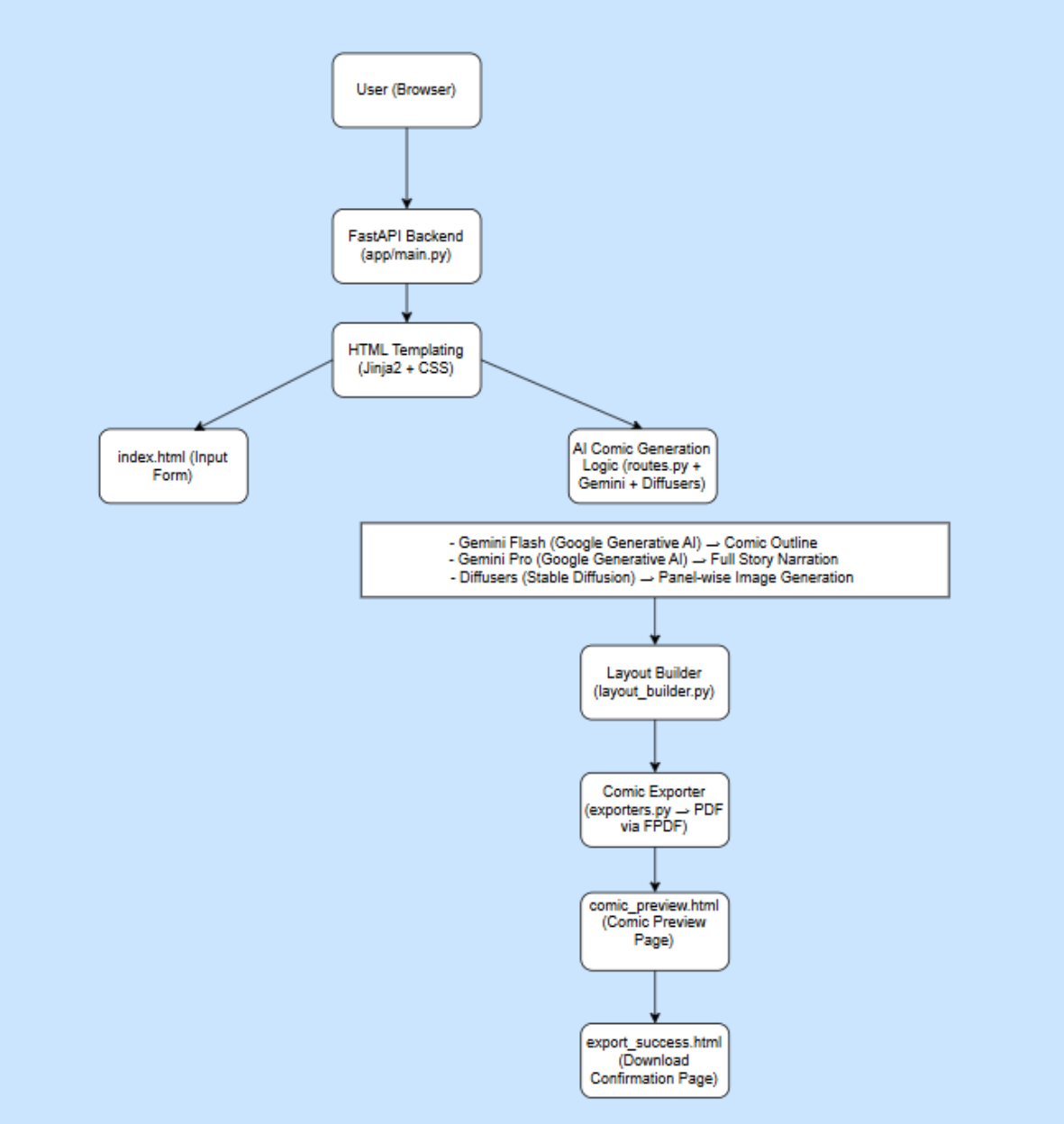
Scenario 3: After reviewing the on-screen preview of their comic, the user decides to download it. The system then:

1. **Layout Binding:** Uses layout_builder.py to assemble each panel's title, image, and text into a structured layout.

2. **PDF Generation:** Calls exporters.py to compile the comic into a **PDF** file using **FPDF**, placing images and narrative text onto consecutive pages and saving the PDF with a timestamped filename.

3. **Export Success:** The user is redirected to an export success page, confirming the successful download. This feature ensures users can save, print, or share their comic in a professional PDF format.

Technical Architecture:



Pre - requisites:

1. **FastAPI Framework Knowledge:** FastAPI Documentation
2. **Uvicorn ASGI Server:** Uvicorn Docs
3. **Python Programming Proficiency:** Python Official Docs
4. **HTML, CSS, and Jinja2 Template Skills:** W3Schools HTML/CSS/Jinja2 Tutorials
5. **Google Gemini API Familiarity:** Google Generative AI Documentation
6. **Hugging Face Diffusers & Stable Diffusion:** Hugging Face Diffusers Documentation
7. **PyTorch Basics:** PyTorch Documentation
8. **PDF Generation with FPDF:** FPDF Documentation
9. **Version Control with Git:**Git Documentation
10. **Python Package & Environment Setup (pip, virtualenv, or conda):** Virtualenv Guide

Project Workflow:

Milestone 1. Model selection and Architecture

- **Activity 1.1. Research and Select the Appropriate Generative AI Model**
- **Activity 1.2. Define the architecture of the application**
- **Activity 1.3. Set up the development Environment**

Milestone 2. Core functionalities Development

- **Activity 2.1. Develop the core functionalities**
- **Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing**

Milestone 3. route.py Development

- **Activity 3.1. Writing the Main Application Logic in routes.py**

Milestone 4. Frontend Development

- **Activity 4.1. Designing and Developing User Interface**
- **Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2**

Milestone 5. Deployment

- **Activity 5.1. Preparing the Application for Local Deployment**
- **Activity 5.2. Testing and Verifying Local Deployment**